

Homework #1

Goal: To make us familiar with Ruby syntax and semantics, enabling us solving basic computational problems.

Hints: Use an IDE for debugging if needed. Start by designing your solution on paper before writing any code. If you're unsure about a specific functionality, feel free to Google it.

Assessed Learning Outcomes: LO 3 (*Implement basic algorithms using Ruby ...*)

Total points: 2%

Tasks:

1. (0.25%) Task #1.

Ask the user to type 3 lines (e.g., before going to the next line the user will hit the 'Enter' key, etc.) on keyboard, and your program should save the lines, appended with line number, to a file named "file.txt". Then, your program also needs to process the file: for each line reports the total number of words that are present.

Below is an example. The user's input is as follows:

```
abc def 123
abc def 456 klmn xyz
abc def
```

Then, myFile.txt gets the following content:

```
L1: abc def 123
L2: abc def 456 klmn xyz
L3: abc def
```

And, the output is as follows. L1: 3, L2: 5, L3: 2

What to submit: Your ruby program as a .rb file with basic documentation. Also, submit a readmeTask1.txt file which shows the output of your program with some example runs.

2. (0.25%) Task #2. Write a function *foo* which takes one English phrase and counts how many times each character appears in the phrase. The function *foo* returns the statistics in the "hash" form as explained with an example below.

An example. Function *foo* gets word "pen and paper" as input, then *foo* returns this hash: $\{p \Rightarrow 3, e \Rightarrow 2, a \Rightarrow 2, r \Rightarrow 1, n \Rightarrow 2, d \Rightarrow 1\}$.

Your program should also have a main function which invokes *foo*.

What to submit: Your ruby program as a .rb file with basic documentation. Also, submit a readmeTask2.txt file which shows the output of your program with some example runs.

3. (0.50%) Task #3. Let the user pick a number (say *x*) between 100 and 200. Now your program simulates tossing a coin *x* times. In particular, your program can contain a loop and in each iteration it randomly makes a choice: head (represented by string 'H') or tail (represented by string 'T'), and stores the outcome ('H' or 'T') in an array. After the iterations, traverse the array and count how many heads and tails were generated. Also, report the ratio of number of heads and number of tails.

What to submit: Your ruby program as a .rb file with basic documentation. Also, submit a readmeTask3.txt file which shows the output of your program with some example runs.

4. (0.75%) Task #4. Go to <http://www.textfiles.com/stories/> and check that this site hosts multiple stories while each story is in a textfile. Download two textfiles of your choice, which have at least 1000 words, and save the files as storyA.txt and storyB.txt. Your program needs to read these files and processes them to collect some statistics. In particular, for each story *x* report the total number of words in *x*, number of unique words in *x*, the 4th-most frequent

word in x and its frequency. Also, your program needs to report the number of unique words that are present in both the files.

What to submit: Your ruby program as a .rb file with basic documentation. Also, submit a readmeTask4.txt file which shows the output of your program with two stories (storyA.txt and storyB.txt). Also, submit your storyA.txt and storyB.txt so that TA can verify the results.

Hints: Use Array and Hash data structures as they are available in Ruby. You may design a regular expression to define a word if needed.

Disclaimer: we did not really check whether this website contains any improper story or language. If you find something improper, please ignore this site and use some other source.